

10



Quelle

I/O

I/O (Input/Output) bezeichnet die Ein- und Ausgabemechanismen, mit denen Programme Daten lesen (Input) und schreiben (Output) können, z. B. über Tastatur, Dateien, Netzwerke oder Bildschirmausgabe.

Java bietet dafür Klassen im Paket **java.io** (blockierend) und **java.nio** (nicht-blockierend).

Blockierend, Nicht-blockierend, Asynchron

Blockierend (I/O) beschreibt, wie ein Programmthread vorgeht: Er **stoppt und wartet aktiv**, bis eine bestimmte Aufgabe (wie das Lesen einer Datei oder das Abrufen eines Netzwerk-Ergebnisses) komplett erledigt ist. Solange er wartet, kann dieser Thread keine anderen Aufgaben erledigen.

Nicht-blockierend (NIO) bedeutet, dass der aufrufende Thread **nicht pausiert**, um auf ein Ergebnis zu warten. Stattdessen gibt er die Kontrolle sofort zurück, nachdem die Aufgabe gestartet wurde. Weil das Ergebnis nicht sofort da ist, muss die Operation fast immer asynchron sein; das Resultat wird dir also auf irgendeine Weise später mitgeteilt, d. h., der Aufrufer wartet nie direkt auf das Ergebnis.

Asynchron bezieht sich darauf, dass eine Operation **im Hintergrund abläuft**, unabhängig vom Hauptprogrammfluss. Wenn eine asynchrone Aufgabe startet, wartet der Aufrufer nicht aktiv auf deren Abschluss. Optional kann auf das Ergebnis gewartet werden (z. B. `get()` oder `join()` bei `CompletableFuture<V>`), was den Thread in diesem spezifischen Moment **blockiert**, bis das Resultat verfügbar ist.

Dateien

Trennzeichen:

Windows: \

Linux: /

Es gibt zwei Arten von Pfadangaben:

Absolut: C:\finanzen\einkommenssteuererklärung2024.txt

Relativ: ./finanzen/einkommenssteuererklärung2024.txt

Es gibt zwei Arten von Dateitypen:

Zeichenorientiert: z. B. yolo.txt, yolo.json, yolo.xml, yolo.yml, yolo.log

Byteorientiert: z. B. unicorn.jpg, unicorn.mp3, unicorn.mp4, unicorn.zip



[illegible]

Byteorientierte Daten verbrauchen **weniger Speicherplatz**, sind **performanter** und **präziser** (keine Typumwandlung in Text nötig). Sie sind für Menschen aber nicht so leicht lesbar.

java.io.File (blockierend)

Früher wurden in Java Dateien durch die Klasse [java.io.File](#) repräsentiert. Man findet noch immer viele Code-Beispiele, die java.io.File verwenden.

```
public static void main(String[] args) {  
    File file = new File("ChuckNorrisFacts.txt");  
  
    // AutoClosable reader (and wrapped objects) will automatically be closed after the try-with-resources-block.  
    try (BufferedReader reader = new BufferedReader(new FileReader(file))) { // wrapped character-based reader  
        String line;  
        while ((line = reader.readLine()) != null) {  
            System.out.println(line);  
        }  
    } catch (IOException e) {  
        System.err.println("Only Chuck Norris can open this file.");  
    }  
}
```



Dateioperationen werfen in Java immer eine **Checked IOException** (oder eine Unterklasse davon)

try-with-resources

Klassen, die das Interface [AutoCloseable](#) implementieren, sollten mit **try-with-resources** verwendet werden (z. B. `ExecutorService`, `BufferedReader`, `HttpClient`).

Java ruft **am Ende** eines **try-with-resources-Blocks** automatisch die **close()**-Methode auf, um die **reservierte Ressource zu schließen** (selbst wenn der try-Block vorzeitig durch `return` oder `throw` beendet wird).

Nicht geschlossene Ressourcen führen bei langlaufenden Programmen zu **Speicherlecks!** 😱

```
public static void main(String[] args) {
    Path filePath = Paths.get("ChuckNorrisFacts.txt");

    // The opened file (reader) will automatically be closed after the try-catch-block.
    try (BufferedReader reader = Files.newBufferedReader(filePath)) {
        String line;
        while ((line = reader.readLine()) != null) {
            System.out.println(line);
        }
    } catch (IOException e) {
        System.err.println("Only Chuck Norris can open this file.");
    }
}
```

```
public interface AutoCloseable {
    void close() throws Exception;
}
```

Merke: Sobald eine Klasse `AutoCloseable` implementiert, verwende `try-with-resources`!

IO-Streams (java.io)



java.io verfügt über unterschiedliche IO-Streams (siehe Tabelle).

IO-Streams sind **keine** Stream<T>.

IO-Streams sind **AutoClosable** und **Einweg-Straßen**, d. h. **entweder** sie **lesen** oder **schreiben**.

IO-Streams lassen sich **verschachteln**, um sie mit **Methoden** und **Funktionalität** zu **erweitern**.

	Byte-based		Character-based	
	Input	Output	Input	Output
Basic (<i>abstract</i>)	<i>InputStream</i>	<i>OutputStream</i>	<i>Reader</i> , <i>InputStreamReader</i>	<i>Writer</i> , <i>OutputStreamWriter</i>
Arrays	ByteArrayInputStream	ByteArrayOutputStream	CharArrayReader	CharArrayWriter
Files	FileInputStream, RandomAccessFile	FileOutputStream, RandomAccessFile	FileReader	FileWriter
Pipes (Threads)	PipedInputStream	PipedOutputStream	PipedReader	PipedWriter
Buffering	BufferedInputStream	BufferedOutputStream	BufferedReader	BufferedWriter
Filtering	FilterInputStream	FilterOutputStream	FilterReader	FilterWriter
Parsing	PushbackInputStream, StreamTokenizer		PushbackReader, LineNumberReader	
Strings			StringReader	StringWriter
Data	DataInputStream	DataOutputStream		
Data - Formatted		PrintStream		PrintWriter
Objects (Serializable)	ObjectInputStream	ObjectOutputStream		
Utilities	SequenceInputStream			

java.nio.file (nicht-blockierend)

Seit JDK 1.7 wurde eine modernere API [java.nio.file](#) eingeführt.

NIO steht für "non-blocking I/O" und bietet mehr Flexibilität, um Dateien zu lesen und zu schreiben.

java.nio.file.Path: Repräsentiert einen Dateipfad (Alternative zu java.io.File).

java.nio.file.Paths: Erzeugt Path-Objekte.

java.nio.file.Files: Utility-Klasse, die Methoden zum Erstellen, Kopieren, Löschen, Lesen und Schreiben von Dateien enthält.

java.nio.file.Path



Path repräsentiert einen Dateipfad. `Paths.get` erstellt einen Path und kümmert sich außerdem automatisch um die korrekte Umwandlung von Dateipfad-Trennzeichen `\` oder `/`.

```
public static void main(String[] args) {  
    Path basePath = Paths.get("C:/Users/sara/Documents"); // Path erstellen  
  
    Path relativePath = Paths.get("Projects/X");  
  
    Path fullPath = basePath.resolve(relativePath); // Pfade kombinieren  
  
    System.out.println("Base:\t" + basePath);  
    System.out.println("Relative:\t" + relativePath);  
    System.out.println("Full:\t" + fullPath);  
  
    // Alternativ: Pfadsegmente direkt hinzufügen  
    Path customPath = basePath.resolve("Games").resolve("Diablo");  
    System.out.println("Custom:\t" + customPath);  
}
```

Ausgabe:

Base:	C:\Users\sara\Documents
Relative:	Projects\X
Full:	C:\Users\sara\Documents\Projects\X
Custom:	C:\Users\sara\Documents\Games\Diablo

Traversal Path

Achtung: Durch unsichere Verkettung von Benutzer-Eingaben mit Strings können Angreifer die Spezielle Zeichkette `..` (bedeutet: gehe ein Verzeichnis hoch bzw. zurück) verwenden, um auf sensible Bereiche des Dateisystems zuzugreifen, die nicht zugänglich sein sollen (**Path-Traversal-Angriff**).

```
public static void main(String[] args) {  
    Path home = Paths.get(System.getProperty("user.home"));  
    System.out.println(home);
```

```
    Path input = Paths.get("../secret/passwords.txt");
```

```
    // normalize() replaces . and .. with actual folders
```



```
    Path path = home.resolve(input).normalize();
```

```
    System.out.println(path);
```



```
    if (!path.startsWith(home)) {  
        throw new SecurityException("Path traversal attack detected: " + path);  
    }
```

```
    System.out.println(path);
```

```
}
```

Ausgabe:

C:\Users\sara

C:\secret\passwords.txt

java.lang.SecurityException



NIO: zeichenorientiert lesen

In diesem Beispiel wird eine Datei gelesen und NIO kümmert sich darum, dass die Datei automatisch geschlossen wird.

```
public static void main(String[] args) {  
    try {  
        Path path = Paths.get("ChuckNorrisFacts.txt");  
        List<String> lines = Files.readAllLines(path); // Ressource wird hier automatisch geschlossen.  
        List<String> lines = Files.readAllLines(path, StandardCharsets.UTF_8); // mit Angabe des Charset.  
        for (String line : lines) {  
            System.out.println(line);  
        }  
        // Files.readAllLines(path).forEach(System.out::println);  
    } catch (IOException e) {  
        System.err.println("Only Chuck Norris can open this file.");  
    }  
}
```

java.nio.Charset

Decimal	Binary	Octal	Hex	ASCII
32	00100000	040	20	SP
33	00100001	041	21	!
34	00100010	042	22	"
35	00100011	043	23	#
36	00100100	044	24	\$
37	00100101	045	25	%
38	00100110	046	26	&
39	00100111	047	27	'
40	00101000	050	28	(
41	00101001	051	29	)
42	00101010	052	2A	*

Dateien bestehen immer aus Bytes (unabhängig davon, ob sie zeichenorientiert oder byteorientiert gelesen oder geschrieben werden).

Ein Charset (Zeichensatz) definiert die Zuordnung von Zeichen zu Bytes und umgekehrt. Beispiele sind [UTF-8](#), ISO-8859-1, Windows-1252, [ASCII](#).

UTF-8 ist plattformunabhängig, unterstützt alle Unicode-Zeichen und ist der de facto Standard für Textdateien.

FYI: Java verwendet intern [UTF-16](#) und konvertiert Zeichen automatisch um.

Unterschiedliche Plattformen und Systeme verwenden unterschiedliche Standardzeichensätze. Ohne die explizite Angabe eines Charsets kann es passieren, dass unterschiedliche Charsets verwendet werden und dadurch Daten falsch interpretiert bzw. unlesbar werden.



NIO: zeichenorientiert lesen mit Stream

In diesem Beispiel kümmert sich NIO nicht darum, die Ressource zu schließen.

"The operating design principle here is '**whoever acquires the resource should release the resource**'. **Files don't auto-close when you read to EOF [End Of File]**; we expect files to be closed explicitly by whoever opened them." [Brian Goetz](#) (Java Language Architect)

```
public static void main(String[] args) {  
    try {  
        Path path = Paths.get("ChuckNorrisFacts.txt");  
        try (Stream<String> lines = Files.lines(path)) { // try-with-resources nötig.  
            try (Stream<String> lines = Files.lines(path, StandardCharsets.UTF_8)) { // mit Angabe des Charset.  
                lines.forEach(System.out::println);  
            }  
        } catch (IOException e) {  
            System.err.println("Only Chuck Norris can open this file.");  
        }  
    }  
}
```

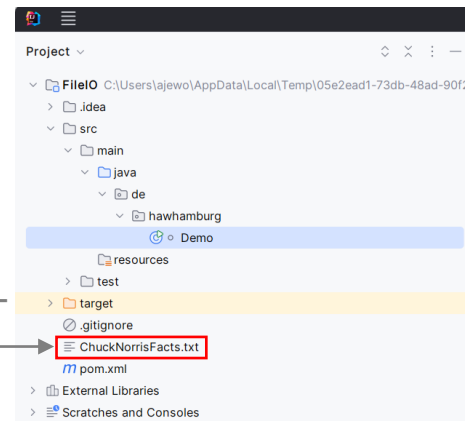


NIO: zeichenorientiert schreiben

In diesem Beispiel kümmert sich die `java.nio.file`-API darum, dass die Datei automatisch geschlossen wird.

```
public static void main(String[] args) {  
    List<String> lines = List.of(  
        "Chuck Norris only uses stunt doubles for crying scenes.",  
        "The universe is not expanding; it is running away from Chuck Norris.",  
        "Chuck Norris doesn't use the steering wheel to drive. The road forms where Chuck Norris wants to go.",  
        "When Chuck Norris falls into the water, he doesn't get wet. The water becomes Chuck Norris."  
    );  
    try {  
        Path path = Paths.get("ChuckNorrisFacts.txt");  
        Files.write(path, lines, StandardOpenOption.CREATE, StandardOpenOption.APPEND);  
    } catch (IOException e) {  
        System.err.println("Only Chuck Norris can write this file.");  
    }  
}
```

Die Datei wird im root-Verzeichnis ("./") des Projekts gespeichert.





NIO: byteorientiert lesen

Bytes werden in NIO mit einem Buffer aus einem FileChannel gelesen.

```
public static void main(String[] args) {  
    Path path = Paths.get("ChuckNorrisFacts.txt");  
    Charset charset = StandardCharsets.UTF_8; // UTF_8 ist bei Dateien der default-Wert, wenn kein Charset angegeben wird.  
    try (FileChannel channel = FileChannel.open(path, StandardOpenOption.READ)) {  
        ByteBuffer buffer = ByteBuffer.allocate(1024);  
        while (channel.read(buffer) != -1) {  
            buffer.flip();  
            CharBuffer charBuffer = charset.decode(buffer);  
            System.out.print(charBuffer);  
            buffer.clear();  
        }  
    } catch (IOException e) {  
        System.err.println("Only Chuck Norris can open this file.");  
    }  
}
```




NIO: byteorientiert schreiben

Bytes werden in NIO mit einem Buffer in einen FileChannel geschrieben.

```
public static void main(String[] args) {  
    List<String> lines = List.of(  
        "Chuck Norris only uses stunt doubles for crying scenes.",  
        "The universe is not expanding; it is running away from Chuck Norris.",  
        "Chuck Norris doesn't use the steering wheel to drive. The road forms where Chuck Norris wants to go.",  
        "When Chuck Norris falls into the water, he doesn't get wet. The water becomes Chuck Norris."  
    );  
  
    Path path = Paths.get("ChuckNorrisFacts.txt");  
    Charset charset = StandardCharsets.UTF_8; // UTF_8 ist bei Dateien der default-Wert, wenn kein Charset angegeben wird.  
    try (FileChannel channel = FileChannel.open(path, StandardOpenOption.CREATE, StandardOpenOption.APPEND)) {  
        for (String line : lines) {  
            byte[] lineBytes = (line + System.lineSeparator()).getBytes(charset);  
            ByteBuffer buffer = ByteBuffer.wrap(lineBytes);  
            channel.write(buffer);  
        }  
    } catch (IOException e) {  
        System.err.println("Only Chuck Norris can write this file.");  
    }  
}
```

Sockets



Sockets

[java.net.Socket](#) ist eine low-level, AutoClosable-Klasse und wird für die **Kommunikation** zwischen zwei oder mehr **Maschinen** über ein **Netzwerk** verwendet. Sockets werden z. B. intern von [HttpClient](#) verwendet.

Man unterscheidet zwischen **Server** (wartet auf eingehende Verbindungen) und **Client** (verbindet sich zum Server).

Eine Socket-Verbindung benötigt eine **IP-Adresse** und einen **Port**. Beispiel:
172.19.192.1:8080 (IPv4)

Die Kommunikation erfolgt über das **TCP**- oder **UDP**-Protokoll.

TCP stellt sicher, dass gesendete Pakete in der richtigen Reihenfolge ankommen und nicht verloren gehen.

ServerSocket: Methoden

ServerSocket()

Erstellt einen ungebundenen ServerSocket.

ServerSocket(int port)

Erstellt einen ServerSocket, der an den angegebenen Port gebunden ist.

void bind(SocketAddress endpoint)

Bindet einen ungebundenen ServerSocket an eine bestimmte Adresse (IP-Adresse und Port). Wird vor accept aufgerufen.

Socket accept()

Wartet (blockierend) auf eine eingehende Verbindung und akzeptiert sie. Gibt ein neues Socket-Objekt zurück, das die Verbindung zum Client darstellt.

ServerSocket: Beispiel

```
class SimpleServer {  
  
    private static final int PORT = 12345;  
  
    public static void main(String[] args) {  
        try (ServerSocket serverSocket = new ServerSocket(PORT);  
            Socket clientSocket = serverSocket.accept();  
            PrintWriter printWriter = new PrintWriter(clientSocket)) {  
  
            printWriter.println("Hi Client!");  
            printWriter.flush();  
        } catch (Exception e) {  
            e.printStackTrace();  
        }  
    }  
  
    private static PrintWriter newPrintWriter(Socket clientSocket) throws IOException {  
        return new PrintWriter(new BufferedOutputStream(clientSocket.getOutputStream()));  
    }  
}
```

Socket: Methoden

Socket(String host, int port)

Erstellt eine Verbindung zu einem bestimmten Host und Port.

Socket(InetAddress host, int port)

Wie zuvor mit InetAddress, eine strictly-typed Repräsentation einer IPv4 oder IPv6-Adresse.

getInputStream()

Gibt den InputStream dieses Sockets zurück. Dieser Stream kann verwendet werden, um Daten vom Server zu lesen.

getOutputStream()

Gibt den OutputStream dieses Sockets zurück. Dieser Stream kann verwendet werden, um Daten an den Server zu schreiben.

close()

Schließt den Socket. Dies beendet die Verbindung und gibt alle zugehörigen Ressourcen frei.

Socket: Beispiel

```
class SimpleClient {  
  
    private static final String IP_ADDRESS = "127.0.0.1";  
    private static final int PORT = 12345;  
  
    public static void main(String[] args) {  
        try (Socket socket = new Socket(IP_ADDRESS, PORT);  
            Scanner scanner = new Scanner(socket.getInputStream())) {  
  
            if (scanner.hasNextLine()) {  
                System.out.printf("Received message: %s%n", scanner.nextLine());  
            } else {  
                System.out.println("No message received from the server.");  
            }  
        } catch (IOException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

Ausgabe:

Received message: Hi Client!

IO-Streams (java.io)

Sockets geben einen **InputStream** oder **OutputStream** zurück, mit den Daten gelesen oder geschrieben werden können. Diese IO-Streams lassen sich **in weitere Klassen verschachteln**, um sie mit **Methoden** und **Funktionalität zu erweitern**.

Die IO-Streams sind **AutoClosable** sowie **Einweg-Straßen**, d. h. entweder sie lesen oder schreiben Daten.

Scanner	PrintWriter	BufferedReader / BufferedWriter	InputStreamReader / OutputStreamWriter	BufferedInputStream / BufferedOutputStream	InputStream / OutputStream
Daten Lesen.	Formatiertes Schreiben.	Lesen / Schreiben. Puffert Zeichen für mehr Effizienz.	Lesen / Schreiben. Brücke zwischen zeichenorientiert und byteorientiert.	Lesen / Schreiben. Brücke zwischen zeichenorientiert und byteorientiert. Puffert Daten für mehr Effizienz.	Lesen / Schreiben.
zeichenorientiert	zeichenorientiert	zeichenorientiert	zeichenorientiert / byteorientiert	byteorientiert	byteorientiert
z. B. Konsole, Dateien, InputStream	z. B. Konsole, Dateien, OutputStream	z. B. Konsole, Dateien, Input/OutputStream	z. B. Dateien, Sockets	z. B. Dateien, Sockets	z. B. Dateien, Sockets
nextLine(), nextInt(), nextDouble(), usw.	print(), println(), printf()	int read() / write(int c)	int read() / write(String)	int read(), read(byte[]) / write(int), write(byte[])	int read(), read(byte[]) / write(int c), write(byte[])
hasNext() / hasNextLine()	flush()	- / flush()	- / flush()	- / flush()	- / flush()
close()	close()	close()	close()	close()	close()

Socket: Server (java.io)

In der Praxis läuft ein Server durchgehend und kann beliebig viele Anfragen akzeptieren.

Jede neue Anfrage wird hierzu asynchron in einem eigenen Thread abgearbeitet.

Ausgabe:

Server is listening on port 12345 ...

22:16:53: ping 1

22:16:54: ping 2

22:16:55: ping 3

22:16:56: ping 4

22:16:57: ping 5

```
class PingServerIO {
    private static final int PORT = 12345;

    public static void main(String[] args) {
        try (ServerSocket serverSocket = new ServerSocket(PORT)) {
            while (true)
                acceptClient(serverSocket);
        } catch (IOException e) { e.printStackTrace(); }
    }

    private static void acceptClient(ServerSocket serverSocket) throws IOException {
        var clientSocket = serverSocket.accept(); // blocks until a client established a new connection.
        Thread.ofVirtual().start(() -> readClientMessage(clientSocket)); // handle new client in a separate thread.
    }

    private static void readClientMessage(Socket clientSocket) {
        String message;
        try (BufferedReader clientReader = new ClientReader(clientSocket)) {
            while ((message = clientReader.readLine()) != null)
                System.out.printf("%s: %s%n", currentTimeStamp(), message);
        } catch (IOException e) { e.printStackTrace(); }
    }

    private static BufferedReader newClientReader(Socket clientSocket) throws IOException {
        return new BufferedReader(new InputStreamReader(clientSocket.getInputStream(), StandardCharsets.UTF_8));
    }

    private static String currentTimeStamp() {
        LocalTime currentTime = LocalTime.now();
        DateTimeFormatter formatter = DateTimeFormatter.ofPattern("HH:mm:ss");
        return currentTime.format(formatter);
    }
}
```

Socket: Client (java.io)

Dieser Client verwendet keinen `PrintWriter`, sondern schreibt die Daten direkt in `BufferedOutputStream`.

Ausgabe:

Sent message to localhost:12345
Sent message to localhost:12345
Sent message to localhost:12345
Sent message to localhost:12345
Sent message to localhost:12345

```
class PingClientIO {

    private static final String IP_ADDRESS = "localhost";
    private static final int PORT = 12345;
    private static final int NUMBER_OF_PINGS = 5;

    public static void main(String[] args) {
        try (Socket server = new Socket(IP_ADDRESS, PORT)) {
            for (int pingCounter = 1; pingCounter <= NUMBER_OF_PINGS; pingCounter++) {
                var message = "ping %d%n".formatted(pingCounter);
                send(server, message);
                System.out.printf("Sent message to %s:%d%n", IP_ADDRESS, PORT);
                Thread.sleep(Duration.ofSeconds(1).toMillis());
            }
        } catch (IOException | InterruptedException e) {
            e.printStackTrace();
        }
    }

    private static void send(Socket server, String message) throws IOException {
        OutputStream outputStream = new BufferedOutputStream(server.getOutputStream());
        outputStream.write(message.getBytes());
        outputStream.flush(); // force to send all written data now.
    }
}
```

Socket: IO vs NIO

IO (blocking I/O):

Verwendet Klassen wie **Socket** und **ServerSocket**, die **blockierende** I/O-Operationen verwenden (z. B. `accept`). Der aktuelle Thread wird blockiert, bis die Operation abgeschlossen ist.

NIO (non-blocking I/O):

Verwendet Klassen wie **SocketChannel** und **ServerSocketChannel**, die **nicht-blockierende** I/O-Operationen **unterstützen**. Der aktuelle Thread wird nicht blockiert und läuft asynchron weiter.

java.nio.channels.Channel

Channel ist eine Klasse, die das **Lesen und Schreiben** aus und/oder in einen **java.nio.Buffer** ermöglicht.

Ein Channel ist eine **Zwei-Wege-Straße**, während IO-Streams in der Regel eine Ein-Wege-Straße sind, und sie können asynchron lesen und schreiben.

Es gibt einige Implementierungen der Channel-Klasse, darunter **FileChannel** für das Lesen und Schreiben im Dateisystem, **DatagramChannel** für das Lesen und Schreiben über ein Netzwerk mit UDP und **SocketChannel** für das Lesen und Schreiben über ein Netzwerk mit TCP.

Fazit

- 1. Modern vs. Legacy (java.nio wins)**
- 2. Ressourcen-Management ist Pflicht**
- 3. Encoding ist kein Zufall**
- 4. Netzwerk & Sockets**
- 5. Security First**

